```
import javax.ws.rs.*;
import javax.ws.rs.core.*;
import javax.annotation.security.*;    #A

@DeclareRoles({"owner", "moderator", "member"})    #B
```

```
@Path("/spaces/{spaceId}/members")
public class SpaceMembersResource {
```

```
    @POST
    @RolesAllowed("owner")    #C
    public Response addMember() { .. }

    @GET
    @RolesAllowed({"owner", "moderator"})    #C
    public Response listMembers() { .. }
}
```

#A Role annotations are in the javax.annotation.security package.
#B Declare roles with the @DeclareRoles annotation.
#C Describe role restrictions with the @RolesAllowed annotation.

The second approach is to retain an explicit notion of lower-level permissions, like those currently used in the Natter API, and to define an explicit mapping from roles to permissions. This can be useful if you want to allow administrators or other users to define new roles from scratch, and it also makes it easier to see exactly what permissions a role has been granted without having to examine the source code of the API. Listing 8.5 shows the SQL needed to define four new roles based on the existing Natter API permissions:

- The social space owner has full permissions.